05 — Databases with PDO

PDO (PHP Data Objects) is the standard way to talk to a database in PHP. Same code for MySQL, PostgreSQL, SQLite…; only the connection string changes.

In this course we use **SQLite**: the whole database is one file, nothing to install. Everything you learn applies as is to MySQL.

Connecting

```php
$pdo = new PDO('sqlite:' . __DIR__ . '/app.db', null, null, [
    PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,  // errors throw exceptions
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,        // rows as associative arrays
]);
```

The same with MySQL / MariaDB:

```php
$pdo = new PDO(
    'mysql:host=127.0.0.1;dbname=shop;charset=utf8mb4',
    'db_user',
    'db_password',
    [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION, PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC]
);
```

The first argument is the **DSN** (Data Source Name). Since PHP 8, `ERRMODE_EXCEPTION` is the default, but setting it explicitly makes the intent clear.

Secure connection, the essentials:

- Never put credentials in a file served publicly; keep them in a config file outside the web root or in environment variables.
- Catch the connection exception and show a generic message: the default error message can reveal the user name and host.

```php
try {
    $pdo = new PDO($dsn, $user, $password, $options);
} catch (PDOException $e) {
    error_log($e->getMessage());       // details in the log, for you
    exit('Database unavailable.');     // generic message, for the visitor
}
```

Running queries

Without user data: `query()` / `exec()`

```php
$pdo->exec('CREATE TABLE IF NOT EXISTS books (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    author TEXT NOT NULL,
    year INTEGER
)');

$books = $pdo->query('SELECT * FROM books ORDER BY title')->fetchAll();
```

With user data: ALWAYS prepared statements

```php
// NEVER do this: SQL injection
$pdo->query("SELECT * FROM users WHERE email = '$email'");
// If $email is   ' OR '1'='1   the query returns every user.
```

A prepared statement sends the query and the values **separately**: the values can never be interpreted as SQL.

```php
// Named placeholders
$stmt = $pdo->prepare('SELECT * FROM books WHERE author = :author AND year > :year');
$stmt->execute(['author' => $author, 'year' => 2000]);
$books = $stmt->fetchAll();

// Positional placeholders
$stmt = $pdo->prepare('SELECT * FROM books WHERE id = ?');
$stmt->execute([$id]);
$book = $stmt->fetch();      // one row, or false if not found
```

Placeholders replace **values** only, not table or column names. For a dynamic `ORDER BY`, check the column against a whitelist.

CRUD

```php
// CREATE
$stmt = $pdo->prepare('INSERT INTO books (title, author, year) VALUES (:title, :author, :year)');
$stmt->execute(['title' => 'Dune', 'author' => 'Frank Herbert', 'year' => 1965]);
$newId = $pdo->lastInsertId();

// READ
$stmt = $pdo->prepare('SELECT * FROM books WHERE id = ?');
$stmt->execute([$newId]);
$book = $stmt->fetch();

// UPDATE
$stmt = $pdo->prepare('UPDATE books SET year = :year WHERE id = :id');
$stmt->execute(['year' => 1966, 'id' => $newId]);
echo $stmt->rowCount();      // number of rows modified

// DELETE
$stmt = $pdo->prepare('DELETE FROM books WHERE id = ?');
$stmt->execute([$newId]);
```

Fetching results

Method	Returns
`fetch()`	the next row, or `false`
`fetchAll()`	all rows, as an array
`fetchColumn()`	the first column of the next row (perfect for `COUNT(*)`)
`fetchAll(PDO::FETCH_CLASS, Book::class)`	objects of your class instead of arrays

```php
$count = $pdo->query('SELECT COUNT(*) FROM books')->fetchColumn();

foreach ($stmt as $row) { /* a statement can be iterated directly */ }
```

`LIKE` search with a prepared statement: the `%` go in the **value**, not in the query.

```php
$stmt = $pdo->prepare('SELECT * FROM books WHERE title LIKE ?');
$stmt->execute(['%' . $search . '%']);
```

Error handling

With `ERRMODE_EXCEPTION`, any SQL error throws a `PDOException`:

```php
try {
    $stmt = $pdo->prepare('INSERT INTO books (title) VALUES (?)');
    $stmt->execute(['Untitled']);       // fails: author is NOT NULL
} catch (PDOException $e) {
    echo 'Error: ' . $e->getMessage();
}
```

Transactions

A transaction groups several queries: **either all succeed, or none is applied**. Classic example: a bank transfer (debit

one account, credit another).

```php
try {
    $pdo->beginTransaction();

    $pdo->prepare('UPDATE accounts SET balance = balance - ? WHERE id = ?')->execute([100, 1]);
    $pdo->prepare('UPDATE accounts SET balance = balance + ? WHERE id = ?')->execute([100, 2]);

    $pdo->commit();              // validate everything
} catch (Throwable $e) {
    $pdo->rollBack();            // cancel everything since beginTransaction()
    throw $e;
}
```

Bonus: pagination, sorting and filtering

As soon as a table grows, you display it **page by page**. In SQL that's `LIMIT` (how many rows) and `OFFSET` (how many to skip):

```sql
SELECT * FROM books ORDER BY title LIMIT 10 OFFSET 20   -- page 3, 10 per page
```

Pagination step by step

```php
$perPage = 10;
$total = (int) $pdo->query('SELECT COUNT(*) FROM books')->fetchColumn();
$pages = max(1, (int) ceil($total / $perPage));   // 25 books -> 3 pages; at least 1

// The page number comes from the URL: validate it, then keep it in range
$page = filter_var($_GET['page'] ?? 1, FILTER_VALIDATE_INT, ['options' => ['min_range' => 1]]) ?: 1;
$page = min($page, $pages);

$offset = ($page - 1) * $perPage;
$stmt = $pdo->prepare('SELECT * FROM books ORDER BY title LIMIT :limit OFFSET :offset');
$stmt->bindValue('limit', $perPage, PDO::PARAM_INT);   // bind as INTEGERS
$stmt->bindValue('offset', $offset, PDO::PARAM_INT);
$stmt->execute();
$books = $stmt->fetchAll();
```

Why `bindValue(..., PDO::PARAM_INT)` instead of `execute([$perPage, $offset])`? `execute()` sends every value as a **string**. Some databases (MySQL) then receive `LIMIT '10'`, which is a syntax error.

Links:

```php
<?php if ($page > 1): ?><a href="?page=<?= $page - 1 ?>">Previous</a><?php endif; ?>
<?php for ($i = 1; $i <= $pages; $i++): ?>
    <a href="?page=<?= $i ?>"><?= $i ?></a>
<?php endfor; ?>
<?php if ($page < $pages): ?><a href="?page=<?= $page + 1 ?>">Next</a><?php endif; ?>
```

Sorting: the one place you can't use a placeholder

Placeholders replace **values**, not column names: `ORDER BY :column` doesn't work. So the column goes into the SQL string, and the only safe way is a **whitelist**:

```php
$sortable = ['title' => 'title', 'author' => 'author', 'year' => 'year', 'price' => 'price'];

$sort = $_GET['sort'] ?? 'title';
$column = $sortable[$sort] ?? 'title';                      // unknown value -> default
$direction = ($_GET['dir'] ?? 'asc') === 'desc' ? 'DESC' : 'ASC';

$sql = "SELECT * FROM books ORDER BY $column $direction, id LIMIT :limit OFFSET :offset";
```

Never write `"ORDER BY " . $_GET['sort']`: that's an SQL injection.

The extra `, id` keeps rows with equal values (two books from 1950) in a stable order, so a book can't appear on two pages.

Filtering

A search adds a `WHERE`, and the **same condition** must be used for the `COUNT(*)` and for the page query, otherwise the

number of pages is wrong:

```php
$where = '';
$params = [];
if ($q !== '') {
    $where = 'WHERE title LIKE :q1 OR author LIKE :q2';    // two names: some drivers refuse reusing one
    $params = ['q1' => "%$q%", 'q2' => "%$q%"];
}
// SELECT COUNT(*) FROM books $where      and      SELECT * FROM books $where ORDER BY ... LIMIT ...
```

Keeping everything in the links

Search, sort and page live in the URL (`?q=asimov&sort=year&dir=desc&page=2`), so each link must change **one** parameter and keep the others. `http_build_query()` builds the query string from an array:

```php
function url(array $changes): string
{
    return '?' . http_build_query(array_merge($_GET, $changes));
}

url(['page' => 3]);                        // keeps q, sort, dir
url(['sort' => 'price', 'page' => null]);  // new sort -> back to page 1 (null keys are dropped)
```

The full working version is the bookshop app of the final project.

Bonus: securing an admin area

The checklist used in the bookshop admin of the final project:

1. **Password stored as a hash** (`password_hash`), checked with `password_verify`.
2. **`session_regenerate_id(true)`** right after a successful login.
3. **Every admin page starts with the check**, before anything else:

   ```php
   if (($_SESSION['admin'] ?? false) !== true) {
       header('Location: login.php');
       exit;          // without exit, the rest of the page still runs!
   }
   ```

4. **Anything that changes data uses POST**, never a link: a link can be triggered by a crawler, a prefetching browser, or an `<img src="...">` on another site.
5. **CSRF token** on every POST form. Another site can make your browser submit a form to your app (with your session cookie!). A random token stored in the session and added to each form proves the form came from your own pages:

   ```php
   // generate once per session
   $_SESSION['csrf'] ??= bin2hex(random_bytes(32));

   // in every form
   <input type="hidden" name="csrf" value="<?= $_SESSION['csrf'] ?>">

   // when processing
   if (!hash_equals($_SESSION['csrf'], $_POST['csrf'] ?? '')) {
       exit('Invalid form token');
   }
   ```

6. **Escape everything** you display, admin pages included.

Organizing the code

Don't scatter `new PDO(...)` everywhere. A common pattern: one function or class that creates the connection once, and **repository** classes that contain the SQL for one table.

```php
class BookRepository
{
    public function __construct(private PDO $pdo) {}

    public function findAll(): array
    {
        return $this->pdo->query('SELECT * FROM books ORDER BY title')->fetchAll();
    }

    public function add(string $title, string $author, ?int $year): int
    {
        $stmt = $this->pdo->prepare('INSERT INTO books (title, author, year) VALUES (?, ?, ?)');
        $stmt->execute([$title, $author, $year]);
        return (int) $this->pdo->lastInsertId();
    }
}
```

This is exactly what frameworks (Symfony with Doctrine, Laravel with Eloquent) do for you, with more automation.

Seeing your SQLite data

Install the VS Code extension **SQLite Viewer** (Florian Klampfer) and click the `.db` file, or use DB Browser for SQLite (Windows, macOS, Linux).

Now do the exercises. Then, to run the same code on a real database server, continue with 06 — MySQL.

05 — Exercises

5.1 — Library (command line)

Create `library.php`, run with `php library.php`.

1. Connect to a SQLite file `library.db` (with the two options `ERRMODE_EXCEPTION` and `FETCH_ASSOC`).
2. Create a `books` table (id, title, author, year) if it doesn't exist.
3. If the table is empty, insert these books with **one** prepared statement executed in a loop:

```php
$books = [
    ['1984', 'George Orwell', 1949],
    ['Dune', 'Frank Herbert', 1965],
    ['The Hobbit', 'J.R.R. Tolkien', 1937],
    ['Neuromancer', 'William Gibson', 1984],
    ['Foundation', 'Isaac Asimov', 1951],
];
```

4. Display all books sorted by year: `1937 - The Hobbit (J.R.R. Tolkien)`.
5. Display the number of books (`COUNT(*)` + `fetchColumn`).
6. Display the books published after 1950 (prepared statement with a placeholder).
7. Update the year of *Dune* to 1966, display the number of modified rows.
8. Delete *Neuromancer*, then display the list again.

Run the script twice: the books must not be inserted twice.

5.2 — Transfer with a transaction

Create `transfer.php`.

1. Create an `accounts` table (id, owner, balance) with two accounts: Alice 500, Bob 100. Recreate it at each run (`DROP TABLE IF EXISTS`) so you always start from the same state.
2. Write a function `transfer(PDO $pdo, int $from, int $to, float $amount): void` that, inside a transaction:
   - checks that the source account has enough money (otherwise throw an exception),
   - debits the source, credits the destination,
   - commits, or rolls back on any error.
3. Test: transfer 200 from Alice to Bob (works), then 1000 from Bob to Alice (fails). Display the balances after each attempt: the failed transfer must not change anything.

5.3 — Final project: Guestbook

Build a small web app, in a folder `guestbook/`, combining everything from the course:

```
guestbook/
├── index.php          list of messages + form to add one
├── admin.php          login (session), then list with "delete" buttons
├── logout.php
└── src/
    ├── bootstrap.php          session_start, autoloader, e() helper — required by every page
    ├── Database.php           returns the PDO connection (creates the table if needed)
    └── MessageRepository.php  findAll(), add(), delete()
```

Requirements:

- `messages` table: id, author, content, created_at.
- **index.php**: form (author, message), server-side validation, errors displayed, values kept, Post/Redirect/Get after saving. Messages displayed newest first, with date formatted `16/10/2026 14:30`. Everything escaped.
- **admin.php**: protected by a password (session + `password_verify`). Each message has a delete button (a small `POST` form with the message id: never delete with a GET link).
- All SQL in `MessageRepository`, all queries prepared when they include data.
- Classes in a namespace (`App\`) and loaded with an autoloader.

Bonus: pagination, 5 messages per page with `LIMIT` / `OFFSET` and *Newer / Older* links. Read the *Bonus: pagination* section of the lesson first. The provided solution includes it.

Next: the final project

Finished? Go to the final project: a bookshop app with search, sorting, pagination, a secured admin and visit statistics. It combines everything from the course.